

hw4

AUTHOR

Andre Gala-Garza

PUBLISHED

April 12, 2026

Disclaimer: Generative AI was used to assist with templating and writing code in this assignment; however, this code was checked manually and edited by hand to ensure accuracy.

Source: OpenAI. (2026). *ChatGPT* [Large language model]. <https://chatgpt.com/>.

```
knitr::opts_chunk$set(echo = TRUE)
```

```
options(
  max.print = 100,
  tibble.print_max = 10,
  tibble.width = Inf
)
```

```
library(tidyverse)
```

— Attaching core tidyverse packages — tidyverse 2.0.0 —

✓ dplyr 1.2.0 ✓ readr 2.1.5

✓ forcats 1.0.1 ✓ stringr 1.5.1

✓ ggplot2 4.0.1 ✓ tibble 3.3.0

✓ lubridate 1.9.4 ✓ tidyr 1.3.1

✓ purrr 1.1.0

— Conflicts — tidyverse_conflicts() —

✗ dplyr::filter() masks stats::filter()

✗ dplyr::lag() masks stats::lag()

ℹ Use the conflicted package (<<http://conflicted.r-lib.org/>>) to force all conflicts to become errors

```
library(dplyr)
```

HPC

Question 1: fun2

```
# running cumulative sum by row
fun2 <- function(mat) {
  n <- nrow(mat)
  k <- ncol(mat)
  ans <- mat
  for (i in 1:n) {
    for (j in 2:k) {
```

```

      ans[i,j] <- mat[i, j] + ans[i, j - 1]
    }
  }
  ans
}

fun2alt <- function(mat) {
  k <- ncol(mat)
  ans <- mat
  for (j in 2:k) {
    ans[, j] <- ans[, j] + ans[, j - 1]
  }
  ans
}

```

```

# Use the functions with this data
set.seed(2315)
dat <- matrix(rnorm(200 * 100), nrow = 200)

```

```

# Check that both functions give the same result
out1 <- fun2(dat)
out2 <- fun2alt(dat)

all.equal(out1, out2)

```

[1] TRUE

```

# Benchmark test
library(microbenchmark)

mb <- microbenchmark(
  original = fun2(dat),
  faster = fun2alt(dat),
  times = 100
)

mb

```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
	original	1881	1976.65	2320.183	2101.7	2374.50	5999.2	100
	faster	272	466.40	641.357	496.1	559.75	11603.5	100

```

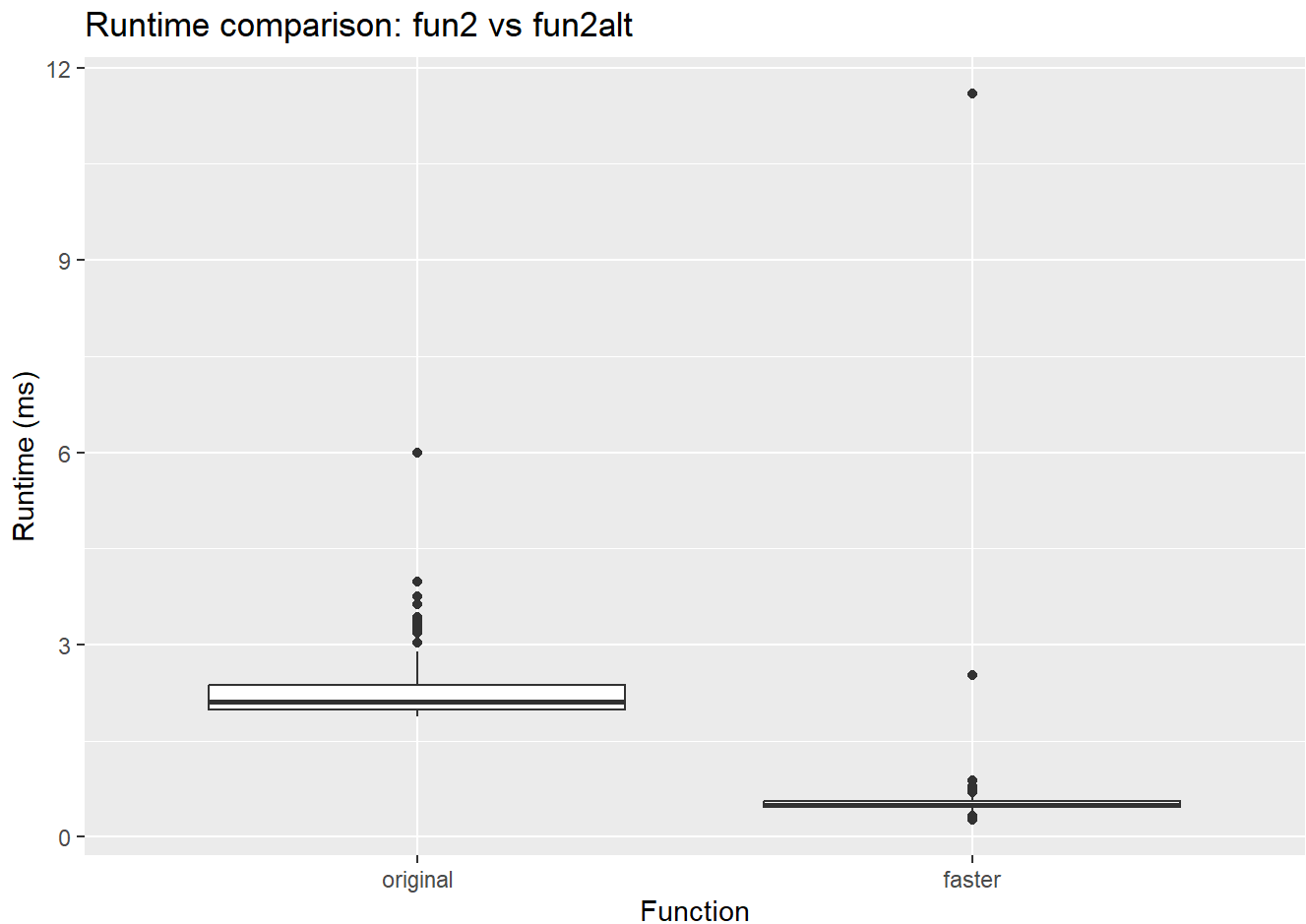
library(ggplot2)

mb_df <- as.data.frame(mb)

ggplot(mb_df, aes(x = expr, y = time / 1e6)) +

```

```
geom_boxplot() +
labs(
  title = "Runtime comparison: fun2 vs fun2alt",
  x = "Function",
  y = "Runtime (ms)"
)
```



fun2alt() is faster because it keeps only one loop and performs each cumulative update on a whole column instead of one matrix element at a time, while still returning exactly the same output as fun2().

Question 2: fun3

```
fun3 <- function(x) {
  out <- numeric(length(x))
  cumsum_x <- 0
  for (i in seq_along(x)) {
    if (x[i] == 0) {
      cumsum_x <- 0
    } else {
      cumsum_x <- cumsum_x + x[i]
    }
  }
  out[i] <- cumsum_x
}
```

```

    }
    out
  }

fun3alt <- function(x) {
  cs <- cumsum(x)
  last_zero <- cummax(seq_along(x) * (x == 0))
  offset <- numeric(length(x))
  nz <- last_zero > 0
  offset[nz] <- cs[last_zero[nz]]
  cs - offset
}

```

```

# Small example for easy checking
x_small <- c(3, 2, 1, 0, 5, 4, 0, 2, 2, 1, 0, 7)

# Larger random example
set.seed(2315)
x_big <- sample(c(0, -2:-1, 1:5), size = 100000, replace = TRUE)

```

```

# Check accuracy
fun3(x_small)

```

```
[1] 3 5 6 0 5 9 0 2 4 5 0 7
```

```
fun3alt(x_small)
```

```
[1] 3 5 6 0 5 9 0 2 4 5 0 7
```

```
all.equal(fun3(x_small), fun3alt(x_small))
```

```
[1] TRUE
```

```
all.equal(fun3(x_big), fun3alt(x_big))
```

```
[1] TRUE
```

```

# Benchmark test
library(microbenchmark)

mb <- microbenchmark(
  original = fun3(x_big),
  faster = fun3alt(x_big),
  times = 100
)

mb

```

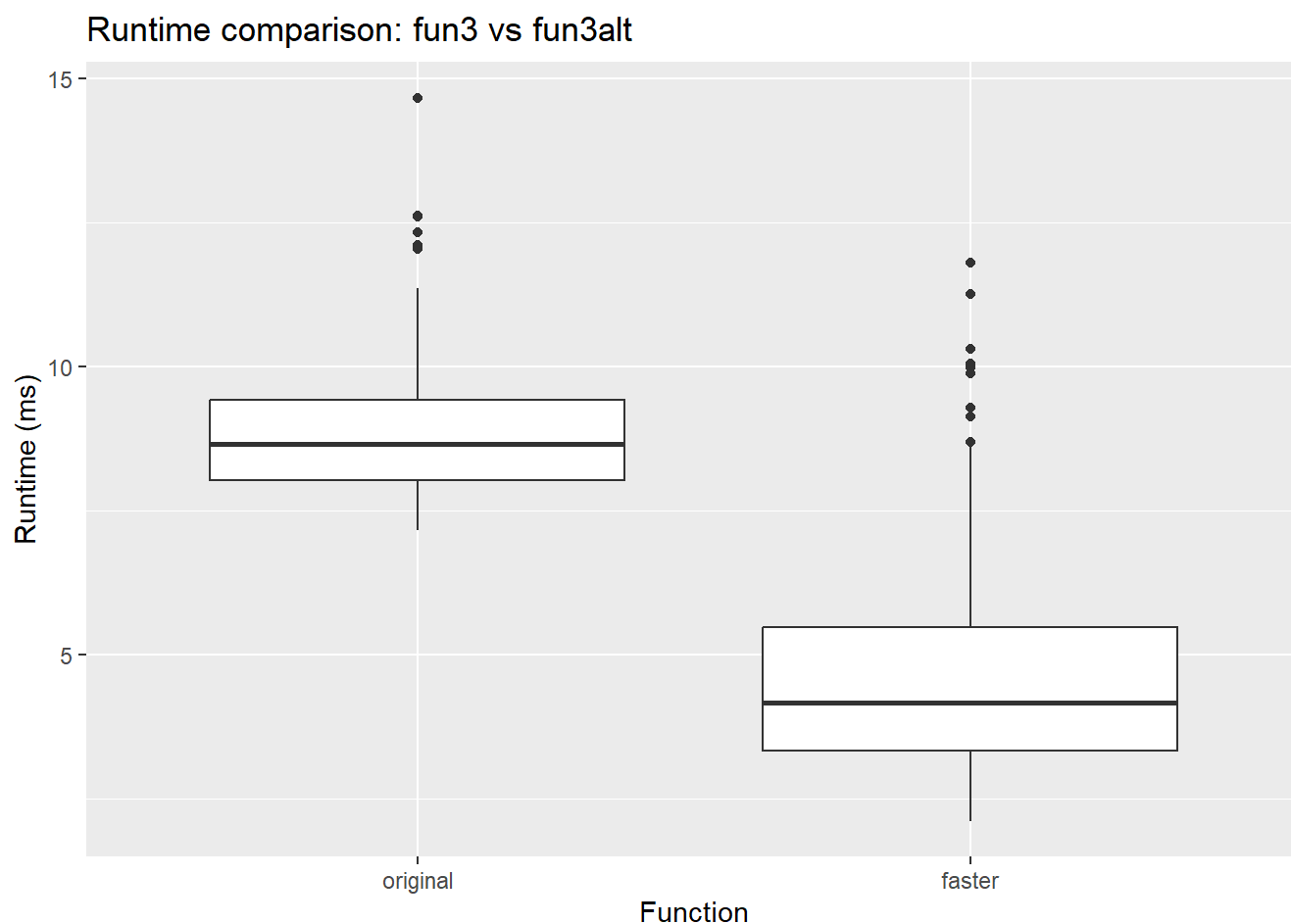
Unit: milliseconds

	expr	min	lq	mean	median	uq	max	neval
original		7.1555	8.01350	8.955787	8.65205	9.43465	14.6680	100
faster		2.1105	3.31445	4.846403	4.16360	5.47345	11.8014	100

```
library(ggplot2)

mb_df <- as.data.frame(mb)

ggplot(mb_df, aes(x = expr, y = time / 1e6)) +
  geom_boxplot() +
  labs(
    title = "Runtime comparison: fun3 vs fun3alt",
    x = "Function",
    y = "Runtime (ms)"
  )
```



fun3alt() is faster because it replaces the element-by-element R loop with vectorized base R operations (cumsum() and cummax()), which do the work in optimized internal code with less interpreter overhead.

Question 3: sim_pi

```
library(parallel)

sim_pi <- function(n = 1000, i = NULL) {
  p <- matrix(runif(n*2), ncol = 2)
  mean(rowSums(p^2) < 1) * 4
}

# Here is an example of the run
set.seed(156)
sim_pi(1000) # 3.132
```

[1] 3.132

```
# Serial version
# This runs the simulation a 4,000 times, each with 10,000 points
set.seed(1231)

serial_time <- system.time({
  ans_serial <- unlist(lapply(1:4000, sim_pi, n = 10000))
  print(mean(ans_serial))
})
```

[1] 3.14124

```
# Parallel version
n_cores <- max(1, detectCores() - 1)
cl <- makeCluster(n_cores)

clusterSetRNGStream(cl, 1231)
clusterExport(cl, "sim_pi")

parallel_time <- system.time({
  ans_parallel <- unlist(parLapply(cl, 1:4000, sim_pi, n = 10000))
  print(mean(ans_parallel))
})
```

[1] 3.1414

```
stopCluster(cl)
```

```
# Visualize that both methods give similar estimates of pi
hist(
  ans_serial,
  breaks = 30,
  freq = FALSE,
  main = "Pi estimates: serial vs parallel",
  xlab = "Estimated pi",
  col = rgb(0, 0, 1, 0.4),
```

```

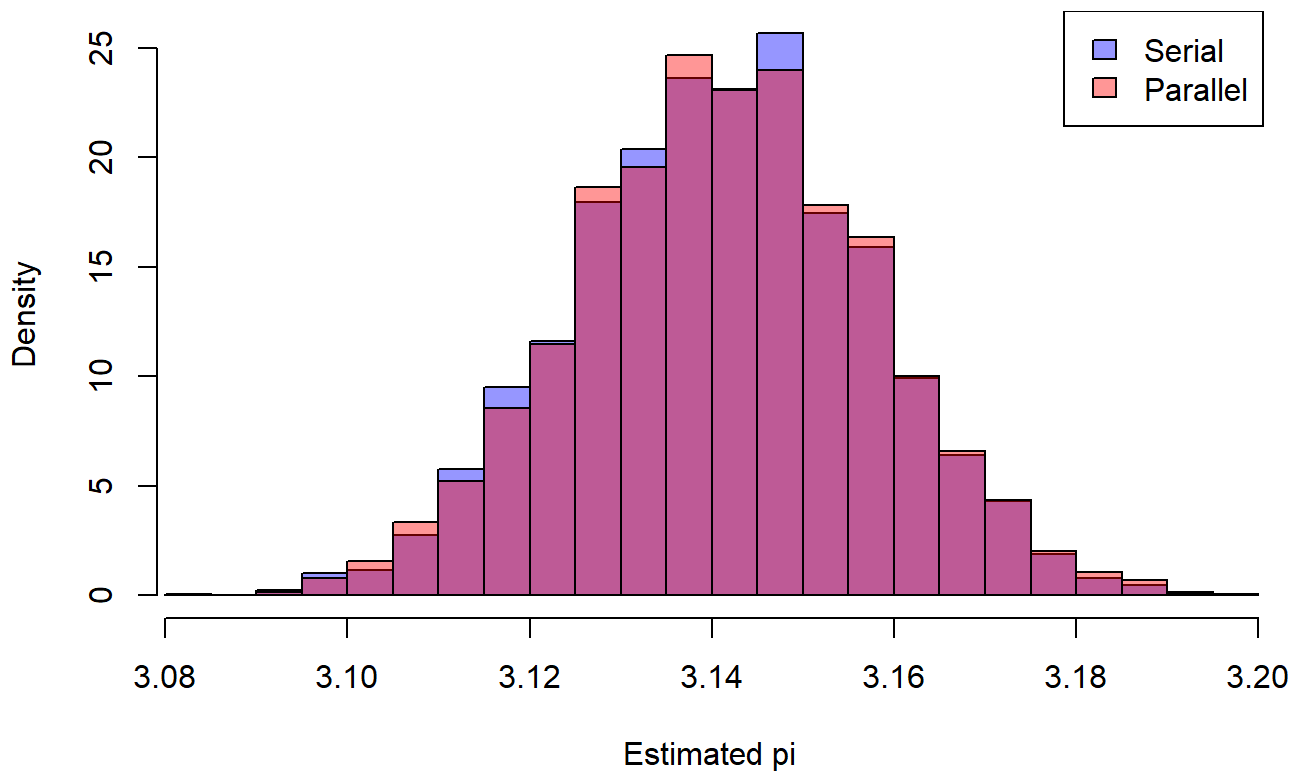
xlim = range(c(ans_serial, ans_parallel))
)

hist(
  ans_parallel,
  breaks = 30,
  freq = FALSE,
  add = TRUE,
  col = rgb(1, 0, 0, 0.4)
)

legend(
  "topright",
  legend = c("Serial", "Parallel"),
  fill = c(rgb(0, 0, 1, 0.4), rgb(1, 0, 0, 0.4))
)

```

Pi estimates: serial vs parallel

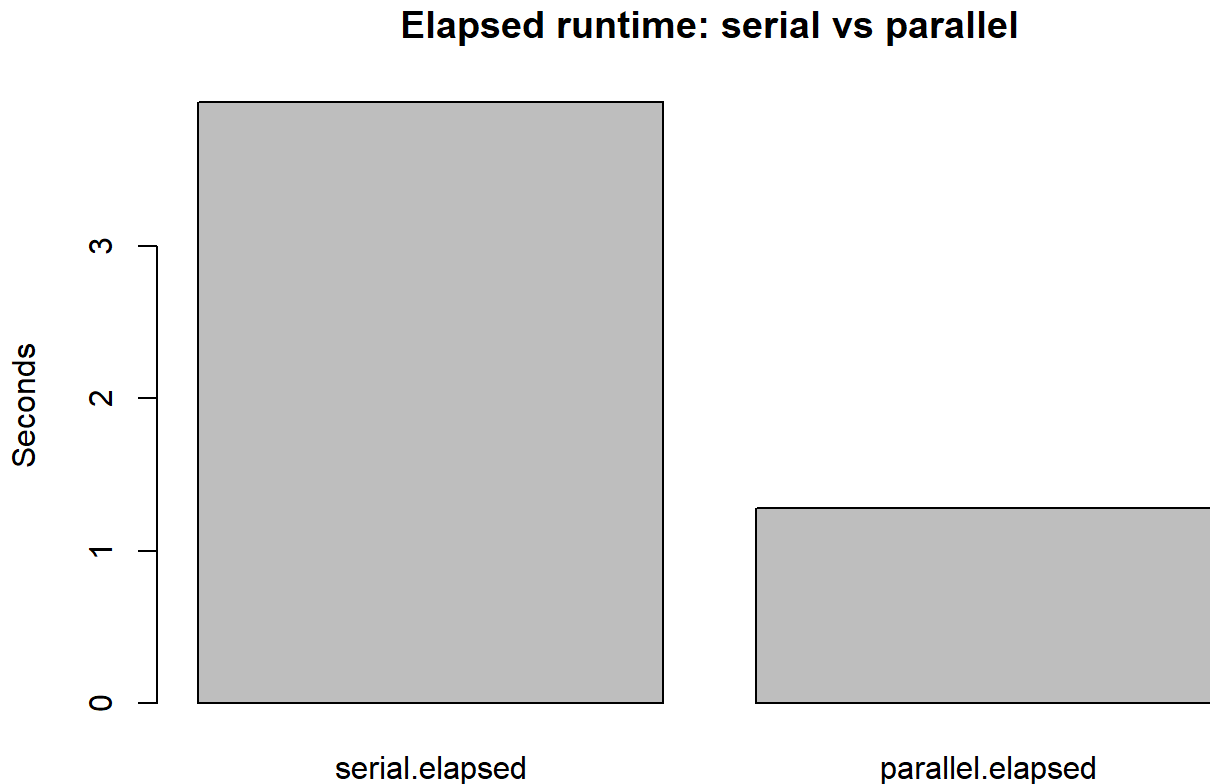


```

# Barplot comparison
runtime_df <- c(
  serial = serial_time["elapsed"],
  parallel = parallel_time["elapsed"]
)

```

```
barplot(  
  runtime_df,  
  main = "Elapsed runtime: serial vs parallel",  
  ylab = "Seconds"  
)
```



The parallel version is faster because it distributes the 4000 independent simulation runs across multiple CPU cores so they can be computed simultaneously instead of sequentially.

SQL

```
# install.packages(c("RSQLite", "DBI", "readr"))  
  
library(RSQLite)  
library(DBI)  
library(readr)  
  
# Initialize a temporary in memory database  
con <- dbConnect(SQLite(), ":memory:")  
  
# Read in tables  
surveys <- read_csv("surveys.csv")
```


Rows: 35549 Columns: 9

— Column specification —

Delimiter: ","

chr (2): species_id, sex

dbl (7): record_id, month, day, year, plot_id, hindfoot_length, weight

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
species <- read_csv("species.csv")
```

Rows: 54 Columns: 4

— Column specification —

Delimiter: ","

chr (4): species_id, genus, species, taxa

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
plots <- read_csv("plots.csv")
```

Rows: 24 Columns: 2

— Column specification —

Delimiter: ","

chr (1): plot_type

dbl (1): plot_id

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Copy data.frames to database
dbWriteTable(con, "surveys", surveys, overwrite = TRUE)
dbWriteTable(con, "species", species, overwrite = TRUE)
dbWriteTable(con, "plots", plots, overwrite = TRUE)
```

```
dbListTables(con)
```

```
[1] "plots" "species" "surveys"
```

Question 4: Surveys Summary

```
dbGetQuery(con,
  "PRAGMA table_info(surveys)"
)
```

	cid	name	type	notnull	dflt_value	pk
1	0	record_id	REAL	0	NA	0
2	1	month	REAL	0	NA	0
3	2	day	REAL	0	NA	0
4	3	year	REAL	0	NA	0
5	4	plot_id	REAL	0	NA	0
6	5	species_id	TEXT	0	NA	0
7	6	sex	TEXT	0	NA	0
8	7	hindfoot_length	REAL	0	NA	0
9	8	weight	REAL	0	NA	0

```
dbGetQuery(con,
  "SELECT
    year,
    month,
    day,
    species_id,
    ROUND(weight / 1000.0, 2) AS weight_kg
  FROM surveys
  WHERE year = 1999
  ORDER BY species_id"
)
```

	year	month	day	species_id	weight_kg
1	1999	1	17	<NA>	NA
2	1999	2	21	<NA>	NA
3	1999	2	21	<NA>	NA
4	1999	2	21	<NA>	NA
5	1999	3	15	<NA>	NA
6	1999	3	15	<NA>	NA
7	1999	3	15	<NA>	NA
8	1999	4	17	<NA>	NA
9	1999	4	18	<NA>	NA
10	1999	4	18	<NA>	NA
11	1999	5	16	<NA>	NA
12	1999	6	13	<NA>	NA
13	1999	9	11	<NA>	NA
14	1999	9	11	<NA>	NA
15	1999	9	11	<NA>	NA
16	1999	9	12	<NA>	NA
17	1999	9	12	<NA>	NA
18	1999	9	12	<NA>	NA
19	1999	10	9	<NA>	NA
20	1999	10	10	<NA>	NA

[reached 'max' / getOption("max.print") -- omitted 1115 rows]

Question 5: Animal Weights

```
# Weights for all animals in 'surveys'
dbGetQuery(con,
  "SELECT
    year,
    month,
    day,
    species_id,
    ROUND(weight / 1000.0, 2) AS weight_kg
  FROM surveys
  WHERE year = 1999
  ORDER BY species_id"
)
```

	year	month	day	species_id	weight_kg
1	1999	1	17	<NA>	NA
2	1999	2	21	<NA>	NA
3	1999	2	21	<NA>	NA
4	1999	2	21	<NA>	NA
5	1999	3	15	<NA>	NA
6	1999	3	15	<NA>	NA
7	1999	3	15	<NA>	NA
8	1999	4	17	<NA>	NA
9	1999	4	18	<NA>	NA
10	1999	4	18	<NA>	NA
11	1999	5	16	<NA>	NA
12	1999	6	13	<NA>	NA
13	1999	9	11	<NA>	NA
14	1999	9	11	<NA>	NA
15	1999	9	11	<NA>	NA
16	1999	9	12	<NA>	NA
17	1999	9	12	<NA>	NA
18	1999	9	12	<NA>	NA
19	1999	10	9	<NA>	NA
20	1999	10	10	<NA>	NA

[reached 'max' / getOption("max.print") -- omitted 1115 rows]

Question 6: Counts and Weights

```
species_year_summary <- dbGetQuery(con, "
  SELECT
    year,
    species_id,
    COUNT(*) AS n_counted,
    AVG(weight) AS avg_weight
  FROM surveys
  GROUP BY year, species_id
  ORDER BY year, species_id
")
```

species_year_summary

	year	species_id	n_counted	avg_weight
1	1977	<NA>	16	NA
2	1977	DM	264	41.141304
3	1977	DO	12	42.666667
4	1977	DS	98	121.437500
5	1977	NL	31	NA
6	1977	OL	10	21.666667
7	1977	OT	17	21.000000
8	1977	OX	7	21.333333
9	1977	PE	6	19.500000
10	1977	PF	31	7.173913
11	1977	PP	7	15.333333
12	1977	RM	2	10.000000
13	1977	SH	1	NA
14	1977	SS	1	NA
15	1978	<NA>	56	NA
16	1978	AH	1	NA
17	1978	DM	389	40.795455
18	1978	DO	22	45.000000
19	1978	DS	320	115.198653
20	1978	NL	48	184.656250
21	1978	OL	47	31.027027
22	1978	OT	66	23.220000
23	1978	OX	1	NA
24	1978	PE	17	20.428571
25	1978	PF	40	7.088235

[reached 'max' / getOption("max.print") -- omitted 510 rows]

Question 7: Number of Species per Taxa

```
taxa_counts <- dbGetQuery(con, "
SELECT
  taxa,
  COUNT(*) AS n_species
FROM species
GROUP BY taxa
HAVING COUNT(*) > 10
ORDER BY n_species DESC
")
```

```
taxa_counts
```

	taxa	n_species
1	Rodent	31
2	Bird	13

Question 8: Number of Animals per Genus

```
genus_plot_counts <- dbGetQuery(con, "
SELECT
  s.plot_id,
  sp.genus,
  COUNT(*) AS n_caught
FROM surveys AS s
INNER JOIN species AS sp
  ON s.species_id = sp.species_id
GROUP BY s.plot_id, sp.genus
ORDER BY s.plot_id ASC, n_caught DESC
")
```

genus_plot_counts

	plot_id	genus	n_caught
1	1	Dipodomys	1330
2	1	Chaetodipus	332
3	1	Onychomys	153
4	1	Neotoma	54
5	1	Perognathus	34
6	1	Peromyscus	28
7	1	Reithrodontomys	24
8	1	Spermophilus	12
9	1	Amphispiza	7
10	1	Ammospermophilus	7
11	1	Pipilo	4
12	1	Sylvilagus	2
13	1	Rodent	1
14	1	Baiomys	1
15	2	Dipodomys	1032
16	2	Chaetodipus	358
17	2	Onychomys	264
18	2	Neotoma	201
19	2	Peromyscus	182
20	2	Reithrodontomys	73
21	2	Sigmodon	23
22	2	Perognathus	21
23	2	Amphispiza	14
24	2	Spermophilus	7
25	2	Ammospermophilus	7
26	2	Sylvilagus	3
27	2	Rodent	1
28	2	Pipilo	1
29	2	Lizard	1
30	2	Callipepla	1
31	2	Calamospiza	1
32	2	Baiomys	1

```
33      3      Chaetodipus      715
[ reached 'max' / getOption("max.print") -- omitted 341 rows ]
```

Question 9: Number of Specimens per Plot

```
species_plottype_counts <- dbGetQuery(con, "
SELECT
  p.plot_type,
  s.species_id,
  sp.genus,
  sp.species,
  COUNT(*) AS n_captured
FROM surveys AS s
INNER JOIN species AS sp
  ON s.species_id = sp.species_id
INNER JOIN plots AS p
  ON s.plot_id = p.plot_id
WHERE s.species_id IS NOT NULL
  AND LOWER(sp.species) <> 'unknown'
GROUP BY p.plot_type, s.species_id, sp.genus, sp.species
ORDER BY p.plot_type, n_captured DESC, s.species_id
")
```

```
species_plottype_counts
```

	plot_type	species_id	genus	species	n_captured
1	Control	DM	Dipodomys	merriami	6760
2	Control	DO	Dipodomys	ordii	1671
3	Control	DS	Dipodomys	spectabilis	1389
4	Control	PP	Chaetodipus	penicillatus	1108
5	Control	OT	Onychomys	torridus	1004
6	Control	PB	Chaetodipus	baileyi	789
7	Control	NL	Neotoma	albigula	610
8	Control	OL	Onychomys	leucogaster	471
9	Control	PF	Perognathus	flavus	424
10	Control	RM	Reithrodontomys	megalotis	403
11	Control	PE	Peromyscus	eremicus	343
12	Control	AH	Ammospermophilus	harrisi	125
13	Control	PM	Peromyscus	maniculatus	120
14	Control	SS	Spermophilus	spilosoma	103
15	Control	AB	Amphispiza	bilineata	76
16	Control	SH	Sigmodon	hispidus	50
17	Control	SA	Sylvilagus	audubonii	29
18	Control	DX	Dipodomys	sp.	27
19	Control	SF	Sigmodon	fulviventer	19
20	Control	CB	Campylorhynchus	brunneicapillus	13

```
[ reached 'max' / getOption("max.print") -- omitted 161 rows ]
```

Question 10: Average Weight of Rodent Species

```
rodent_avg_weight <- dbGetQuery(con, "
SELECT
  s.species_id,
  sp.genus,
  sp.species,
  AVG(s.weight) AS avg_weight
FROM surveys AS s
INNER JOIN species AS sp
  ON s.species_id = sp.species_id
WHERE sp.taxa = 'Rodent'
GROUP BY s.species_id, sp.genus, sp.species
ORDER BY avg_weight DESC
")

rodent_avg_weight
```

	species_id	genus	species	avg_weight
1	NL	Neotoma	albigula	159.245660
2	DS	Dipodomys	spectabilis	120.130546
3	SS	Spermophilus	spilosoma	93.500000
4	SH	Sigmodon	hispidus	73.148936
5	SF	Sigmodon	fulviventer	58.878049
6	SO	Sigmodon	ochrognathus	55.414634
7	DO	Dipodomys	ordii	48.870523
8	DM	Dipodomys	merriami	43.157864
9	PB	Chaetodipus	baileyi	31.735943
10	OL	Onychomys	leucogaster	31.575258
11	PH	Perognathus	hispidus	31.064516
12	OT	Onychomys	torridus	24.230556
13	PE	Peromyscus	eremicus	21.586508
14	PM	Peromyscus	maniculatus	21.364155
15	OX	Onychomys	sp.	21.000000
16	PI	Chaetodipus	intermedius	19.250000
17	PL	Peromyscus	leucopus	19.138889
18	PX	Chaetodipus	sp.	19.000000
19	PP	Chaetodipus	penicillatus	17.173942
20	RX	Reithrodontomys	sp.	15.500000
21	RF	Reithrodontomys	fulvescens	13.386667
22	RM	Reithrodontomys	megalotis	10.585010
23	RO	Reithrodontomys	montanus	10.250000
24	BA	Baiomys	taylori	8.600000
25	PF	Perognathus	flavus	7.923127

[reached 'max' / getOption("max.print") -- omitted 4 rows]

Question 11: Number of Animals per Genus (ggplotly)

```
library(dplyr)
library(ggplot2)
library(plotly)
```

Attaching package: 'plotly'

The following object is masked from 'package:ggplot2':

last_plot

The following object is masked from 'package:stats':

filter

The following object is masked from 'package:graphics':

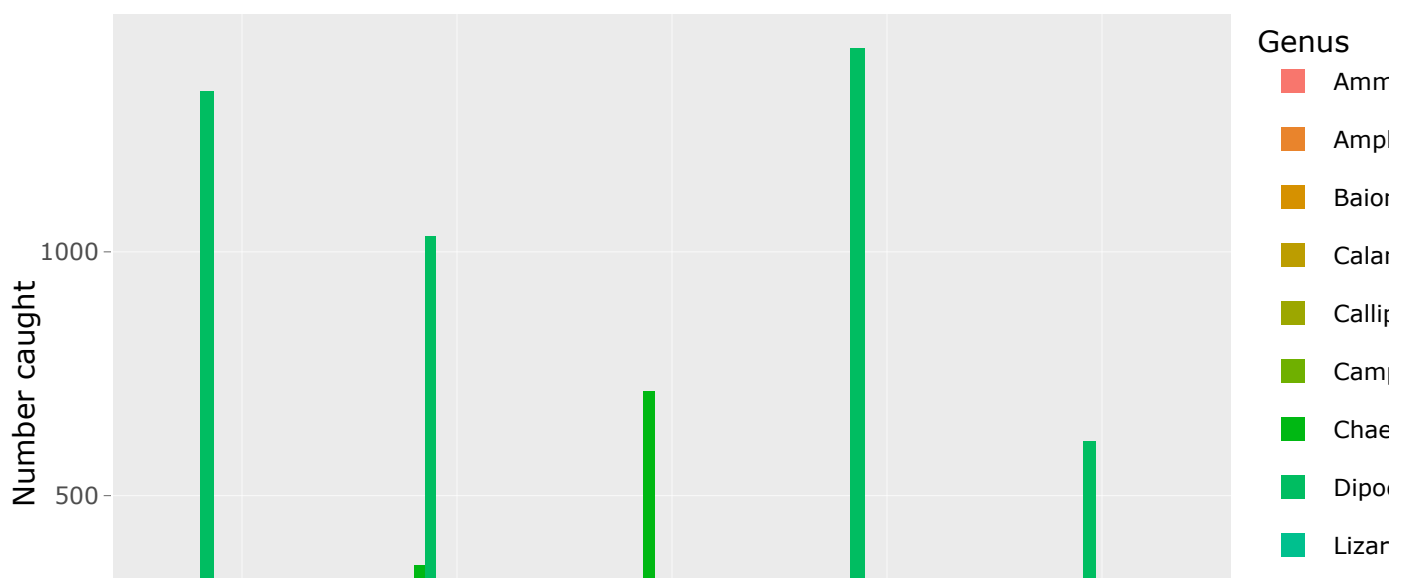
layout

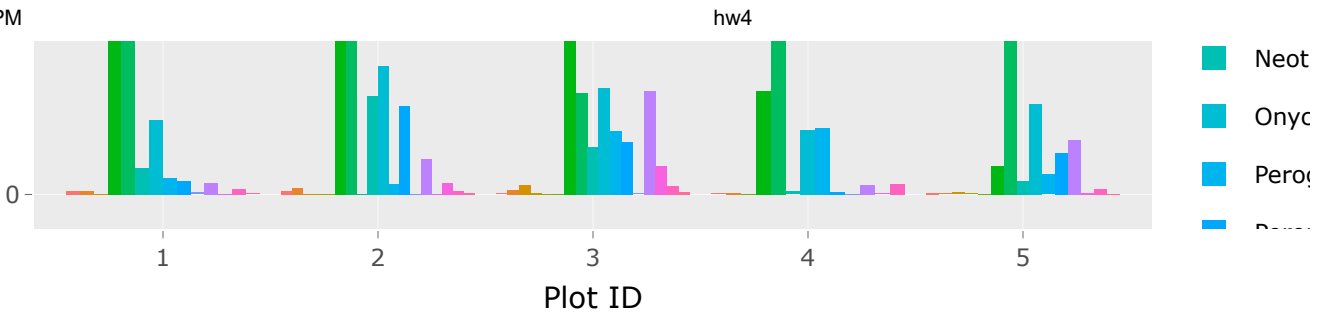
```
genus_plot_counts_1_5 <- genus_plot_counts %>%
  filter(plot_id %in% 1:5)

p <- ggplot(genus_plot_counts_1_5, aes(x = factor(plot_id), y = n_caught, fill = genus)) +
  geom_col(position = "dodge") +
  labs(
    title = "Number of animals caught by genus for plots 1-5",
    x = "Plot ID",
    y = "Number caught",
    fill = "Genus"
  )

plotly::ggplotly(p)
```

Number of animals caught by genus for plots 1-5





Question 12: Counts and Weights (plotly)

```
library(dplyr)
library(plotly)

species_year_plot <- species_year_summary %>%
  filter(!is.na(avg_weight))

plot_ly(
  data = species_year_plot,
  x = ~year,
  y = ~avg_weight,
  color = ~species_id,
  type = "scatter",
  mode = "lines+markers",
  hovertemplate = paste(
    "Year: %{x}",
    "<br>Species: %{color}",
    "<br>Average weight: %{y:.2f}",
    "<extra></extra>"
  )
) %>%
  layout(
    title = "Average species weight over time",
    xaxis = list(title = "Year"),
    yaxis = list(title = "Average weight")
  )
```

Warning in RColorBrewer::brewer.pal(max(N, 3L), "Set2"): n too large, allowed maximum for palette Set2 is 8

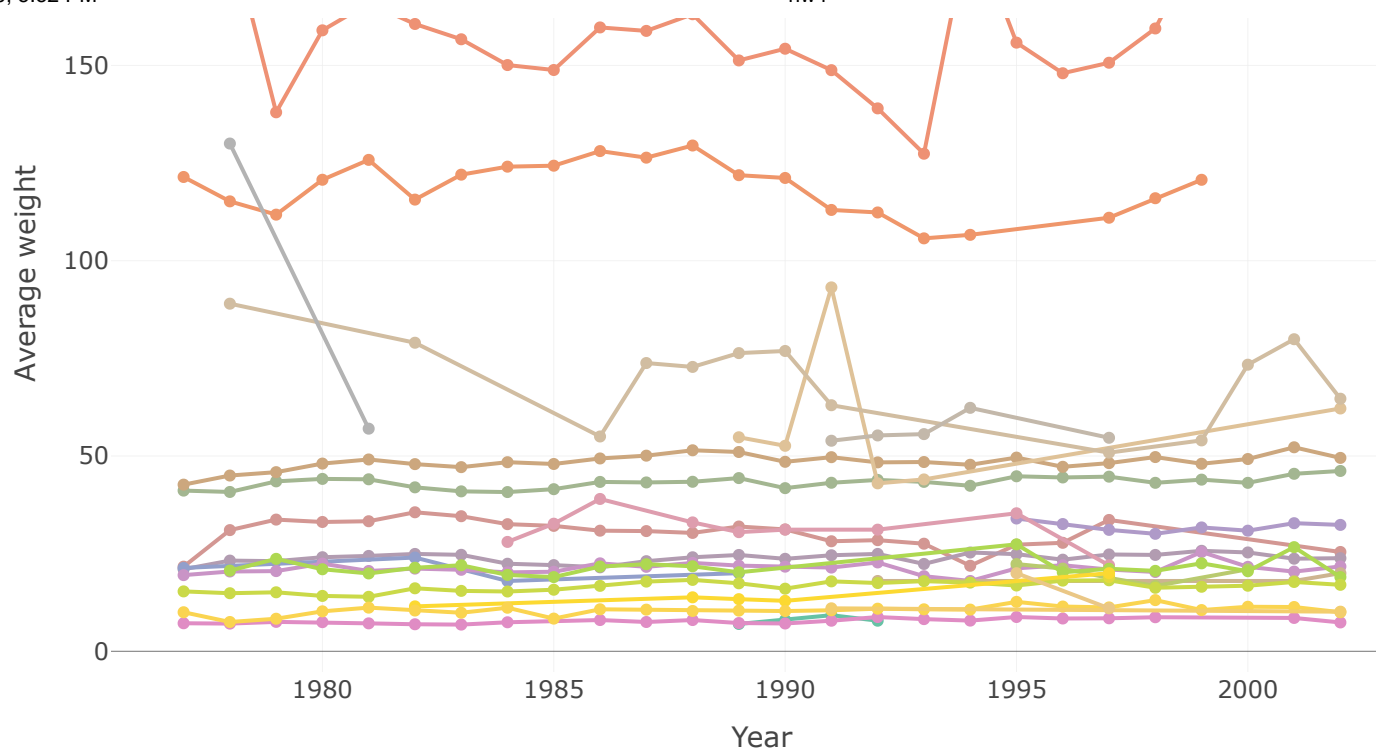
Returning the palette you asked for with that many colors

Warning in RColorBrewer::brewer.pal(max(N, 3L), "Set2"): n too large, allowed maximum for palette Set2 is 8

Returning the palette you asked for with that many colors

Average species weight over time





One potential solution to remove noise in this plot is to keep only species that appear in at least 10 years:

```
species_year_plot_clean <- species_year_summary %>%
  filter(!is.na(avg_weight)) %>%
  group_by(species_id) %>%
  filter(n() >= 10) %>%
  ungroup()

plot_ly(
  data = species_year_plot_clean,
  x = ~year,
  y = ~avg_weight,
  color = ~species_id,
  type = "scatter",
  mode = "lines+markers",
  hovertemplate = paste(
    "Year: %{x}",
    "<br>Species: %{color}",
    "<br>Average weight: %{y:.2f}",
    "<extra></extra>"
  )
) %>%
  layout(
    title = "Average species weight over time (species with at least 10 years of data)",
    xaxis = list(title = "Year"),
    yaxis = list(title = "Average weight")
  )
```

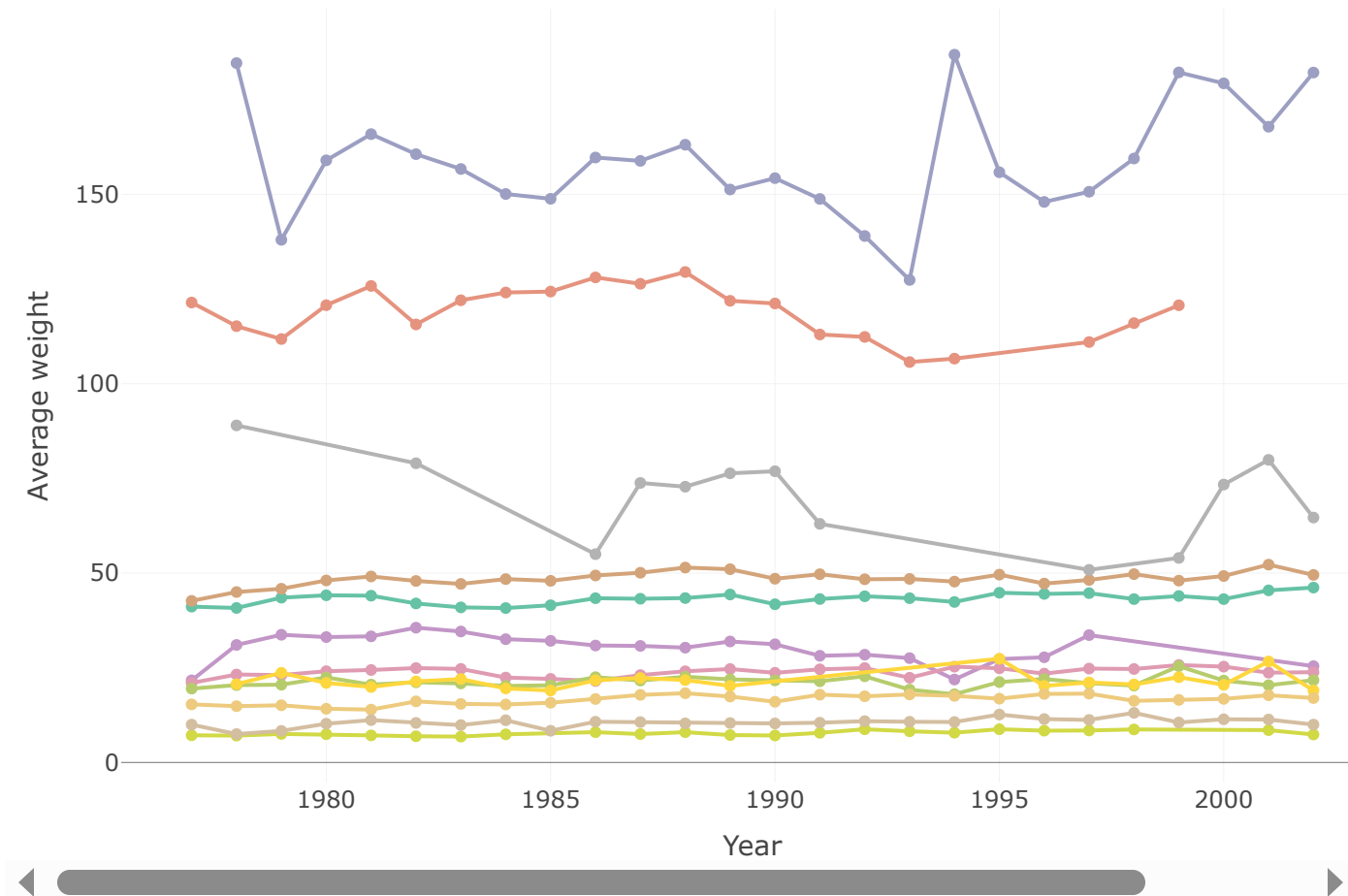
Warning in RColorBrewer::brewer.pal(max(N, 3L), "Set2"): n too large, allowed maximum for palette Set2 is 8

Returning the palette you asked for with that many colors

Warning in RColorBrewer::brewer.pal(max(N, 3L), "Set2"): n too large, allowed maximum for palette Set2 is 8

Returning the palette you asked for with that many colors

Average species weight over time (species with at least 10 years of data)



We can also remove the markers and keep only the lines in the plot:

```
plot_ly(
  data = species_year_plot_clean,
  x = ~year,
  y = ~avg_weight,
  color = ~species_id,
  type = "scatter",
  mode = "lines",
  hovertemplate = paste(
    "Year: %{x}",
    "<br>Species: %{color}",
    "<br>Average weight: %{y:.2f}",
    "<extra></extra>"
  )
) %>%
layout(
  title = "Average species weight over time",
  xaxis = list(title = "Year"),
```

```
yaxis = list(title = "Average weight")  
)
```

Warning in RColorBrewer::brewer.pal(max(N, 3L), "Set2"): n too large, allowed maximum for palette Set2 is 8

Returning the palette you asked for with that many colors

Warning in RColorBrewer::brewer.pal(max(N, 3L), "Set2"): n too large, allowed maximum for palette Set2 is 8

Returning the palette you asked for with that many colors

Average species weight over time

